

EX. NO:7**BINARY SEARCH TREE****DATE:****QUESTION 7:**

Implement creation, insertion, searching and traversals operations (preorder, inorder, postorder) in the Binary Search Tree.

AIM:

To implement creation, insertion, searching and traversals operations (preorder, inorder, postorder) in the Binary Search Tree.

ALGORITHM:

Step 1: Start the program.

Step 2: Define the node with data, left, and right pointers.

Step 3: Initialize the tree by setting root = NULL.

Step 4: Display the menu and read the user's choice.

Step 5: If the choice is **Insert**:

- Create a new node using create(value).
- If root == NULL, set root = newNode.
- Otherwise, compare value with temp->data.
- If value < temp->data, move to temp->left; otherwise move to temp->right.
- Insert the new node at the appropriate position.

Step 6: If the choice is **Search**:

- Set temp = root.
- If temp->data == value, the value is found.
- If value < temp->data, move to the left child.
- Otherwise, move to the right child.
- Continue until the value is found or temp == NULL.

Step 7: If the choice is **Inorder**:

- Perform inorder(root).
- Traverse as **Left** → **Root** → **Right**.

Step 8: If the choice is **Preorder**:

- Perform preorder(root).
- Traverse as **Root** → **Left** → **Right**.

Step 9: If the choice is **Postorder**:

- Perform postorder(root).
- Traverse as **Left** → **Right** → **Root**.

Step 10: If the choice is **Exit**, terminate the program.

Step 11: Repeat Steps 4–10 until the user selects **6 (Exit)**.

Step 12: Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left;
    struct node *right;
};

struct node *root = NULL;
struct node *temp, *newnode, *parent;

struct node* create(int value) {
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->left = NULL;
    newnode->right = NULL;
    return newnode;
}

void insert(int value) {
    newnode = create(value);
    if (root == NULL)
    {
        root = newnode;
    }
    return;
}

temp = root;
parent = NULL;
```

```
while (temp != NULL)
{
parent = temp;
if (value < temp->data)
{
temp = temp->left;
}
else
{
temp = temp->right;
}
}
if (value < parent->data)
parent->left = newnode;
else
parent->right = newnode;
}

struct node* search(int value) {
temp = root;
while (temp != NULL)
{
if (temp->data == value)
return temp;
else if (value < temp->data)
temp = temp->left;
else
temp = temp->right;
}
return NULL;
}

void inorder(struct node *root) {
if (root != NULL)
{
inorder(root->left);
printf("%d ", root->data);
inorder(root->right);
}
```

```
}  
}  
void preorder(struct node *root) {  
    if (root != NULL)  
    {  
        printf("%d ", root->data);  
        preorder(root->left);  
        preorder(root->right);  
    }  
}  
void postorder(struct node *root) {  
    if (root != NULL)  
    {  
        postorder(root->left);  
        postorder(root->right);  
        printf("%d ", root->data);  
    }  
}  
int main() {  
    int choice, value;  
    struct node *result;  
    while (choice != 6)  
    {  
        printf("\nBinary Search Tree\n");  
        printf("1. Insert\n");  
        printf("2. Search\n");  
        printf("3. Inorder\n");  
        printf("4. Preorder\n");  
        printf("5. Postorder\n");  
        printf("6. Exit\n");  
        printf("Enter choice: ");  
        scanf("%d", &choice);  
        switch (choice) {  
        case 1:  
            printf("Enter value: ");  
            scanf("%d", &value);
```

```
insert(value);
break;
case 2:
printf("Enter value to search: ");
scanf("%d", &value);
result = search(value);
if (result != NULL)
printf("Value found.\n");
else
printf("Value not found.\n");
break;
case 3:
printf("Inorder Traversal: ");
inorder(root);
printf("\n");
break;
case 4:
printf("Preorder Traversal: ");
preorder(root);
printf("\n");
break;
case 5:
printf("Postorder Traversal: ");
postorder(root);
printf("\n");
break;
case 6:
printf("Exit.\n");
break;
default:
printf("invalid!\n");
}
}
return 0;
}
```

OUTPUT:

```
[ita2536@mritlin harshan]$ vi ex7.c
[ita2536@mritlin harshan]$ cc ex7.c
[ita2536@mritlin harshan]$ ./a.out
Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 1

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 2

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 3

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 4
```

```
Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 5

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 2
Enter value to search: 3
Value found.

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 3
Inorder Traversal: 1 2 3 4 5

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 4
Preorder Traversal: 1 2 3 4 5
```

```
Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 5
Postorder Traversal: 5 4 3 2 1

Binary Search Tree
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 6
Exit.

[ita2536@mritlin harshan]$
```

RESULT:

Thus, the implementation of creation, insertion, searching and traversals operations (preorder, inorder, postorder) in the Binary Search Tree was executed successfully.